

UTTERTUNE: LORA-BASED TARGET-LANGUAGE PRONUNCIATION EDIT AND CONTROL IN MULTILINGUAL TEXT-TO-SPEECH

Shuhei Kato

Independent Researcher, Tokyo, Japan

ABSTRACT

We propose **UtterTune**, a lightweight adaptation method that fine-tunes a multilingual text-to-speech (TTS) system based on a large language model (LLM) architecture, designed to enhance the controllability of pronunciation in a target language while preserving performance in others. While LLM architectures have enabled TTS models to achieve remarkable naturalness, accurately modeling grapheme-to-phoneme (G2P) mapping and prosody remains challenging, especially when the model omits an explicit G2P module and directly processes minimally encoded text (e.g., byte-pair encoding). UtterTune leverages low-rank adaptation to enable the control of segmental pronunciation and pitch accent at the phoneme level for Japanese speech, the target language in this paper, while maintaining naturalness and speaker similarity in a zero-shot setting. Objective and subjective evaluations confirm its effectiveness.

Index Terms— text-to-speech, large language model, low-rank adaptation, pronunciation, controllability

1. INTRODUCTION

Text-to-speech (TTS) models based on large language model (LLM) architecture (LLM-TTS in this paper) have demonstrated exceptional naturalness, especially in zero-shot multi-speaker and multilingual synthesis, leading the way in speech synthesis technology [1, 2, 3, 4]; however, reproducing accurate pronunciation remains challenging. Some multilingual LLM-TTS, such as CosyVoice 2 [4], are designed to take raw text (characters) as input and tokenize it via byte-pair encoding (BPE) [5], without explicit phonemic or prosody markers. This design contrasts with conventional neural sequence-to-sequence TTS, which typically converts input text into phonemes (grapheme-to-phoneme; G2P) and prosody information, if needed, using a text frontend before feeding it into the model [6, 7, 8, 1]. Omitting the text frontend simplifies model architecture and facilitates multilingual training. On the other hand, such models require a large amount of speech-text pairs that cover the diversity of the target language because they predict segmental pronunciation and prosody data-driven. This dependency engenders known difficulties with some languages.

For example, the model occasionally cannot accurately predict Japanese segmental pronunciation and pitch accent, resulting in degraded speech quality and intelligibility. The

reasons why data-driven G2P is challenging in Japanese include the fact that Japanese is written using a combination of *kanji* (Chinese characters; ideograms), and *kana* (phonograms), the use of more than 2,000 kanji in everyday language [9], and the inclusion of over 6,000 kanji in the Japanese Industrial Standards [10]. Many kanji have two or more readings, and many specific combinations of kanji have their special readings. Since Japanese and Chinese share the same tokens for common kanji, Chinese pronunciations may appear in the synthesized speech of Japanese. From the perspective of pitch accent, accents are determined individually for each word, and the point of pitch falling (accent nuclei) may shift when words are linked [11, 12].

Users of conventional Japanese TTS can easily correct pronunciation errors by modifying the output of the text frontend. However, in cases like CosyVoice2, where G2P and accent estimation are performed implicitly, fixing errors is not straightforward. Segmental pronunciation can be corrected by using kana in the input text, but users lack the ability to control the accent.

To address this issue, we propose UtterTune, a method that enables intuitive pronunciation control for multilingual LLM-TTS models without requiring full fine-tuning or re-training from scratch. Our approach leverages low-rank adaptation (LoRA) [13]. LoRA is an efficient fine-tuning technique originally introduced for adapting LLMs by injecting small trainable weight matrices (low-rank updates) into the Transformer layers while keeping the original weights frozen. It drastically reduces the number of trainable parameters and avoids catastrophic forgetting of the base model’s capabilities. Several papers have applied LoRA to TTS for speaker adaptation and style control [14, 15, 16, 17]. UtterTune applies LoRA to the CosyVoice 2 model, with a specific focus on segmental pronunciation and pitch accent control in Japanese as the initial target. By fine-tuning only a small set of additional parameters on Japanese speech-transcription pair data, UtterTune enables editing and control to read input texts with accurate pronunciation, including native-like pitch accent. This approach does not affect the model’s performance on other languages because the LoRA module is not required.

Our contributions are summarized as follows: 1) We demonstrate the first application of LoRA to a multilingual LLM-TTS for language-specific segmental pronunciation and prosody edit and control, introducing a minimal intervention (only two special tokens, described later, and LoRA layers).

2) UtterTune preserves naturalness and speaker similarity in a zero-shot setting, evidenced by objective and/or subjective metrics. 3) The adaptation is highly parameter-efficient: The LoRA updates form a small file, less than 0.5% compared to the base model. The LoRA weights are available via <https://github.com/shuheikatoinfo/UtterTune>.

2. TOKYO JAPANESE PRONUNCIATION AND ITS NOTATION

In this paper, we adopt Japanese, especially Tokyo Japanese, as the target language for the proposed method; therefore, we briefly describe the pronunciation of Tokyo Japanese and its notation.

Segmental pronunciation of Tokyo Japanese speech can be written in kana, a set of phonograms. Kana is divided into *hiragana* and *katakana*. Both are completely interchangeable in terms of pronunciation. The number of hiragana or katakana characters is approximately 80 each [10, 18]. Most of the kana characters represent one mora. A single-mora sound can comprise multiple phonemes, e.g., a consonant and a subsequent vowel.

In terms of prosody, pitch accent is one of the most essential elements in Tokyo Japanese speech. In speech processing, pitch accent in Tokyo Japanese is often expressed as a binary value, indicating whether the mora is high (H) or low (L) [12, 19, 6]. A single accentual unit is called an accent phrase. An accent phrase contains at most one accent nucleus. Accent phrases generally begin with a low-high (LH) pattern, with the pitch falling from high to low starting from the mora where the accent nucleus is located. As an exception, if the accent nucleus is at the first mora, accent phrases begin with a high-low (HL) pattern. Each word has its unique accent pattern, and there is a minimal tendency for which words have which accent patterns [11]. When words combine, the accent nucleus may shift, and there are shift patterns to some extent [12]. Accents can help distinguish between words and can also indicate semantic cohesion [11, 12].

A standard notation for Tokyo Japanese pronunciation exists for speech synthesis [20]. In this standard, segmental pronunciation can be represented using katakana, with accent nuclei indicated by apostrophes (') and accent phrase boundaries indicated by slashes (/).

3. PROPOSED METHOD

3.1. Base Model

We build UtterTune on CosyVoice 2, a state-of-the-art multilingual LLM-TTS system. CosyVoice 2 deals with speech as discrete speech tokens encoded by a semantic codec (a vector quantizer). During synthesis, a unified text-speech language model (LM; Qwen2.5-0.5B [21]) generates these tokens from input text. The generated tokens are then passed to a vocoder to produce a waveform. Text input is processed by a BPE tokenizer (shared across languages) instead of a conventional phoneme-based text front-end. This design enables the model to operate end-to-end on raw text. On the other hand, it also

requires the model to learn pronunciation data-driven. The CosyVoice 2 LM is trained in an autoregressive next-token prediction manner. The LM treats the text and subsequent speech tokens in a single sequence. While effective in many cases, this approach has limitations for Japanese, as described in the introduction. Indeed, the CosyVoice 2 paper reports that the model’s performance on Japanese lags behind that of other languages (e.g., English, Chinese) [4].

3.2. LoRA Integration in Transformer Layers

To add pronunciation controllability of Japanese to CosyVoice 2 without full fine-tuning, we utilize LoRA [13]. LoRA inserts trainable low-rank matrices into the Transformer’s weight structure, typically by decomposing the large weight matrices of each layer’s query, key, value, and output (Q, K, V, and O) projections in the multi-head self-attention mechanism. We freeze all the original weights of CosyVoice 2 and add LoRA layers to every self-attention block in the text-speech LM. Concretely, for each dense weight matrix $W \in \mathbb{R}^{d \times k}$ in the Q/K/V/O linear transformations, we introduce a pair of smaller matrices B and C of rank r (with $r \ll \min(d, k)$). During training, the effective weight becomes $W + \alpha \cdot BC$, where α is a scaling factor and BC has rank r . Only the B and C parameters and embeddings for new tokens, as described in the following subsection, are updated during training. LoRA dramatically reduces the number of trainable parameters, less than 0.5% of the base model. This parameter efficiency not only saves GPU memory and training time, but also tends to preserve the base model’s knowledge, since the pre-trained weights remain untouched.

3.3. Phoneme Inputs with Special Tokens

Another key component of UtterTune is intuitive input that guides the model’s segmental pronunciation and pitch accent. CosyVoice 2 initially uses raw text (kanji, kana, etc., in the case of Japanese). The raw text doesn’t always expressly represent segmental pronunciation and pitch accent.

We extend the model’s input representation by adding two special tag tokens, `<PHON_START>` and `<PHON_END>`, to the vocabulary. When the model encounters a `<PHON_START>` token, it indicates that the subsequent sequence represents phonemes (segmental pronunciation; regarding kanas as phonemes here for simplicity) and accent rather than ordinary text, until the `<PHON_END>` token is reached. We fine-tune the model such that it can accept inputs of the form: e.g., `<PHON_START>チ'ミ/モ一リヨ一<PHON_END>` to synthesize speech of “魑魅魍魎” (“evil spirits and monsters”). For the model training, following the standard [20], we convert a portion of the input sentences to katakana, with accent nuclei indicated by apostrophes (') and accent phrase boundaries indicated by slashes (/), and wrap them in the new tag tokens. This teaches the model to map phonemic sequences with accent information to the correct speech tokens. We keep the original embedding matrix frozen. Only the embeddings of the new tag tokens, which are randomly initialized, are updated during fine-tuning. This approach

is analogous to adding a “mode switch” for the model: In normal mode (no tags), it is expected to read text as before; in phoneme mode, it follows a supplied spelling.

4. EXPERIMENTS

4.1. Training and Validation Data

For adaptation, we used two public speech corpora: JSUT [22], a female single-speaker corpus, and JVS [23], a multi-speaker corpus comprising 100 speakers. We utilized the “basic5000” and “voiceactress100” sets from JSUT, and the “parallel100” set from JVS. Randomly-selected one noun from each transcript of these corpora was converted into phonemic sequences for the `<PHON_START> ... <PHON_END>` format described in 3.3, using `pyopenjtalk` [6], a publicly available Japanese G2P tool. The author, a native speaker, then modified estimation errors.

The training process thus pairs these transcripts with the original speech recordings. As a result, the dataset comprised approximately 15,097 Japanese speech-transcription pairs. We held out 754 utterances from it for validation.

4.2. Evaluation Data

For evaluation, we prepared two test sets: 1) a general test set and 2) a targeted segmental pronunciation and pitch accent controllability stress-test set.

Test Set 1 was based on 50 Japanese sentences, each 20 to 40 characters in length, including punctuation marks. The sentences were obtained by instructing ChatGPT o3 [24] to generate easy-to-read Japanese sentences because CosyVoice 2 sometimes mispronounces sentences in public speech corpora. After the generation, the author reviewed all the generated sentences and made necessary grammatical or semantic modifications. Using these sentences as input, we performed zero-shot synthesis using prompt audio and transcription derived from ReasonSpeech v2.0 [25], a 35,000-hour Japanese speech corpus collected from television broadcasts. We chose 48 audio clips, each clip representing one speaker, using a signal-to-noise ratio estimated by WADA-SNR [26] of 25 dB or higher and lasting 4 seconds or less. We then performed voice activity detection (VAD) using `torchaudio.functional.vad` [27] on these audio clips and used 42 audio clips and their transcription as the prompts for which speech segments were detected. We also applied the same VAD to the synthesized speech because we observed that CosyVoice 2 occasionally generates non-speech sounds before or after speech segments. As a result, we obtained 2,100 stimuli per system.

Test Set 2 consisted of 50 sentences designed to assess the controllability of segmental pronunciation and pitch accent. The sentences were obtained by instructing ChatGPT o3 to generate easy-to-read Japanese sentences that include one difficult-to-read word each because no appropriate public benchmarks existed. After the generation, the author also reviewed all the generated sentences and made necessary modifications. The difficult words lasted from 2 to 6 morae, and the accent nuclei were located from the first to

the last morae. For the proposed method, the pronunciation of the target difficult-to-read word was spelled out in the `<PHON_START> ... <PHON_END>` format. We performed zero-shot synthesis on the same speakers used in Test Set 1. As a result, we obtained 2,100 stimuli per system.

4.3. Fine-Tuning Configuration

We adopted the pre-trained CosyVoice 2-0.5B checkpoint [28] and applied our LoRA modifications. The LoRA rank, scaling alpha, and dropout rate were set to 16, 64, and 0.05, respectively, for all injected layers. We trained using the AdamW [29] optimizer with a learning rate of 1×10^{-4} . We used a warming-up schedule for the first 10% of training steps and a cosine decay. The mini-batch size was set to 8, and we fine-tuned for 20,000 steps on an NVIDIA RTX 4090 GPU in less than one hour.

The only parameters updated were the LoRA layers and the embeddings for `<PHON_START>` and `<PHON_END>`. We did not apply any auxiliary losses; the training objective remained the cross-entropy on next-token prediction for speech tokens given the text prompt.

4.4. Baseline Systems

We employed the official CosyVoice 2-0.5B checkpoint [28] as the baseline. For Test Set 2, we added a kana-input baseline, where we replaced transcription with katakana for the target difficult-to-read word in the input texts.

4.5. Evaluation Metrics

Automatic naturalness mean opinion score (MOS) prediction

We utilized UTMOSv2, a non-intrusive model that predicts the MOS of a given speech sample [30]. Samples with UTMOSv2 less than 2.0 in any system were excluded from the tabulation to exclude the effect of extremely inferior quality samples due to synthesis errors. The ratio of dropped samples was less than 2% for each test set and system.

Naturalness MOS

We conducted a subjective listening test where participants listened to and scored the naturalness of synthesized speech on a 5-point MOS scale. 16 crowdworkers who are native Japanese speakers participated in this test. We gathered MOS scores for 126 stimuli paired for each model, stratified by UTMOSv2 scores from Test Set 1, with a minimum of four ratings per sample. The stimuli were selected to contain all 42 speakers.

Objective speaker similarity (SS)

Although our focus is on pronunciation controllability, we want to ensure the adapted model does not degrade the ability to maintain the target speaker’s voice characteristics. We measure speaker similarity using ERes2Net [31, 32]. Speaker embeddings were extracted from synthesized speech and the corresponding prompt speech of the same speaker. We then compute the cosine similarity between these embeddings.

Test Set	System	UTMOSv2 $\uparrow$	MOS $\uparrow$	SS $\uparrow$	CER $\downarrow$	Accent correctness $\uparrow$
1	CosyVoice 2-0.5B (baseline)	3.25	3.44	0.693	0.0557	-
	Proposed	3.28	3.88	0.696	0.0639	-
2	CosyVoice 2-0.5B (baseline)	3.27	-	0.695	0.1005	-
	CosyVoice 2-0.5B (kana input)	3.28	-	0.692	0.0595	0.472
	Proposed	3.30	-	0.697	0.0626	0.975

Table 1. Experimental results. Bold text indicates values that were significantly ($p < 0.05$) better for each test set based on the one-sided paired Wilcoxon test (and Bonferroni correction if needed).

Character error rate (CER)

To quantify intelligibility, we used an automatic speech recognition (ASR) system to transcribe the synthesized speech and calculated CER against the input text. We employed OpenAI’s Whisper large-v3 [33] to transcribe the outputs. Considering that Test Set 2 contains difficult-to-read words, Whisper was forced to output only kana and limited symbols (U+3000 to U+30FF in Unicode [34]). We then removed all spaces and symbols from the transcription and converted it into katakana for CER calculation. The same procedure was used for Test Set 1. Some prompt speech has been observed to leak to the beginning of the synthesized speech, and the leakage portion was excluded from the CER calculation. Samples with CER > 0.5 were omitted from aggregation to avoid serious ASR errors.

Accent correctness

We performed a subjective accent evaluation on Test Set 2. In this paper, due to time constraints, the author carefully listened to each synthesized speech and judged whether the pitch accent pattern of the target word followed the instruction. Since this metric required highly specialized manual work, we limited the speakers to four out of 42. The four speakers were chosen as those closest to the centroids of four clusters formed by k-means clustering, using uniform manifold approximation and projection [35] to project ERes2Net embeddings (192 dimensions) extracted from the prompt audio of the 42 speakers onto two dimensions.

5. RESULTS AND DISCUSSION

5.1. Naturalness and Speaker Similarity

Table 1 summarizes the objective and subjective scores. On Test Set 1, UtterTune increased the crowd-sourced MOS from 3.44 to 3.88. A one-sided paired Wilcoxon test confirmed the gain was statistically significant ($p < 0.0001$). Considering the results of UTMOSv2 together, the results at least showed that the proposed method does not degrade naturalness.

For Test Set 2, UTMOSv2 scores had no significant differences among systems, and the 99% confidence interval (CI) of the difference scores was less than 0.06 for all combinations of systems. The three systems can be considered equivalent.

Across both test sets, the ERes2Net cosine similarity hovered around 0.69 for all systems and showed no significant differences. This means that UtterTune kept the ability to maintain the voice characteristics in the prompt speech.

5.2. Effect of Phoneme Tags and Kana Transcription

Comparing the results of the baseline and kana input systems, feeding kana transcription alone reduced CER from 0.1005 to 0.0595. CosyVoice 2 could pronounce words segmentally correctly once kanji ambiguity was resolved. Yet its accent correctness was low (0.472): The model often read the proper phonemes with an incorrect pitch pattern.

UtterTune, by contrast, attained 0.975 accent correctness while keeping a competitive CER (0.0626). The difference of CER to the kana-baseline was significant ($p < 0.005$), but the effect size was negligible (Cliff’s delta = 0.115 [36]). The LoRA adaptation learnt not only to obey the given phoneme string but also to place the pitch fall at the right mora.

To confirm that the effect was not extended to the parts not enclosed by `<PHON_START>` and `<PHON_END>`, an additional 50 sentences containing only one katakana word with an accented nucleus were prepared based on the ChatGPT o3 generation. If affected, these words should be synthesized with a flat pitch pattern. The sentences were synthesized by the four speakers using the baseline and the proposed method. The percentage of correct accents, scored by the author, was 0.925 (baseline) and 0.905 (proposed method), with no significant difference, and a 99% CI of [-0.080, 0.045]. This means that UtterTune did not leak pronunciation controllability outside the tag tokens.

6. CONCLUSION

We proposed UtterTune, a lightweight LoRA-based adaptation that endows a multilingual LLM-TTS model with phoneme-level pronunciation control. By freezing the 0.5-B-parameter CosyVoice 2 backbone and fine-tuning only rank-16 LoRA matrices plus two special input tokens, UtterTune requires less than 0.5% additional parameters and a few GPU-hours of training. It raised naturalness from 3.44 to 3.88 MOS, kept CER below 7% on everyday sentences, and boosted accent correctness from 0.472 to 0.975 on stress-test material without degrading speaker similarity. UtterTune never deteriorates in the performance of other languages in principle because LoRA does not need to be applied when synthesizing other languages. These results demonstrate that minor, targeted updates can rectify language-specific deficiencies—here, Japanese G2P and pitch accent—while preserving the strong performance of a multilingual model.

Future research includes applying UtterTune to other Japanese dialects and other languages, as well as verifying its performance during code switching.

7. REFERENCES

- [1] C. Wang *et al.*, “Neural codec language models are zero-shot text to speech synthesizers,” arXiv:2301.02111 [cs.CL], 2023.
- [2] E. Kharitonov *et al.*, “Speak, read and prompt: High-fidelity text-to-speech with minimal supervision,” *Trans. Assoc. Comput. Linguistics*, vol. 11, pp. 1703–1718, 2023.
- [3] S. Chen *et al.*, “VALL-E 2: Neural codec language models are human parity zero-shot text to speech synthesizers,” arXiv:2406.05370 [cs.CL], 2024.
- [4] Z. Du *et al.*, “CosyVoice 2: Scalable streaming speech synthesis with large language models,” arXiv:2412.10117 [cs.SD], 2024.
- [5] P. Gage, “A new algorithm for data compression,” *C Users J.*, vol. 12, no. 2, pp. 23–38, 1994.
- [6] R. Yamamoto *et al.*, “pyopenjtalk,” 2018. [Online]. Available: <https://github.com/r9y9/pyopenjtalk>
- [7] T. Hayashi *et al.*, “JVS RECIPE,” 2020. [Online]. Available: <https://github.com/espnet/espnet/tree/master/egs2/jvs/tts1>
- [8] J. Kim, J. Kong, and J. Son, “Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech,” in *Proc. Int. Conf. Mach. Learn.*, 2021.
- [9] P. M. of Jpn., “Joyo kanji-hyo,” 2010. [Online]. Available: https://www.bunka.go.jp/kokugo_nihongo/sisaku/joho/joho/kijun/naikaku/pdf/joyokanjihyo_20101130.pdf
- [10] Minister of Int. Trade and Industry, “JIS X0208,” 1978.
- [11] NHK Broadcast. Culture Res. Inst., Ed., *NHK’s New Dictionary of Japanese Pronunciation and Accentuation*. NHK Publishing, 2016.
- [12] Y. Sagisaka and H. Sato, “Accentuation rules for Japanese word concatenation,” *The IEICE Trans. (Japanese Ed.)*, vol. J66-D, no. 7, pp. 849–856, 1983.
- [13] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” arXiv:2106.09685 [cs.CL], 2021.
- [14] C.-P. Hsieh, S. Ghosh, and B. Ginsburg, “Adapter-based extension of multi-speaker text-to-speech model for new speakers,” in *Proc. INTERSPEECH*, 2023, pp. 3028–3032.
- [15] H. Lou *et al.*, “StyleSpeech: Parameter-efficient fine tuning for pre-trained controllable text-to-speech,” in *Proc. ACM Int. Conf. Multimedia in Asia*, 2024.
- [16] H. Kim *et al.*, “VoiceTailor: Lightweight plug-in adapter for diffusion-based personalized text-to-speech,” in *Proc. INTERSPEECH*, 2024, pp. 4413–4417.
- [17] Ł. Bondaruk and J. Kubiak, “LoRP-TTS: Low-rank personalized text-to-speech,” arXiv:2502.07562 [cs.SD], 2025.
- [18] Int. Org. Standardization, “ISO/IEC 10646:2020/Amd 1:2023,” 2023.
- [19] K. Tokuda, K. Oura, and S. Sako, “Open JTalk version 1.11,” 2018. [Online]. Available: <https://open-jtalk.sourceforge.net/>
- [20] Jpn. Electronics and Inf. Technol. Industries Assoc., “JEITA IT-4006,” 2010.
- [21] B. Hui *et al.*, “Qwen2.5-coder technical report,” arXiv:2409.12186 [cs.CL], 2024.
- [22] R. Sonobe, S. Takamichi, and H. Saruwatari, “JSUT corpus: Free large-scale Japanese speech corpus for end-to-end speech synthesis,” arXiv:1711.00354 [cs.CL], 2017.
- [23] S. Takamichi *et al.*, “JVS corpus: Free Japanese multi-speaker voice corpus,” arXiv:1908.06248 [cs.SD], 2019.
- [24] OpenAI, “OpenAI o3 and o4-mini.” [Online]. Available: <https://help.openai.com/en/articles/9624314-model-release-notes#h.cfb262abff>
- [25] Reason Human Interaction Lab, “reazonspeech,” 2024. [Online]. Available: <https://huggingface.co/datasets/reason-research/reazonspeech>
- [26] C. Kim and R. M. Stern, “Robust signal-to-noise ratio estimation based on waveform amplitude distribution analysis,” in *Proc. INTERSPEECH*, 2008, pp. 2598–2601.
- [27] “torchaudio.functional.vad,” 2024. [Online]. Available: <https://docs.pytorch.org/audio/2.7.0/generated/torchaudio.functional.vad.html>
- [28] Inst. Intell. Comput., “CosyVoice2-0.5B.” [Online]. Available: <https://www.modelscope.cn/models/iic/CosyVoice2-0.5B>
- [29] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Representations*, 2019.
- [30] K. Baba *et al.*, “The T05 system for the VoiceMOS Challenge 2024: Transfer learning from deep image classifier to naturalness MOS prediction of high-quality synthetic speech,” in *Proc. IEEE Spoken Lang. Technol. Workshop*, 2024.
- [31] Y. Chen *et al.*, “An enhanced Res2Net with local and global feature fusion for speaker verification,” in *Proc. INTERSPEECH*, 2023.
- [32] “iic/speech_eres2net_sv_zh-cn_16k-common.” [Online]. Available: https://www.modelscope.cn/models/iic/speech_eres2net_sv_zh-cn_16k-common
- [33] OpenAI, “whisper-large-v3,” 2023. [Online]. Available: <https://huggingface.co/openai/whisper-large-v3>
- [34] Unicode, Inc., “Unicode® 16.0.0.” [Online]. Available: <https://www.unicode.org/versions/Unicode16.0.0/>
- [35] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform manifold approximation and projection for dimension reduction,” arXiv:1802.03426 [stat.ML], 2020.
- [36] N. Cliff, “Dominance statistics: Ordinal analyses to answer ordinal questions,” *Psychological Bull.*, vol. 114, no. 3, pp. 494–509, 1993.